# ReasoningFlow with Tool Calls and Web Search

*Incremental Annotation of Streaming Reasoning Content*

Nathan Conklin
nathan.conklin@vt.edu
Department of Computer Science,
Virginia Tech

## Abstract

This design applies ReasoningFlow to a chat interaction in which `reasoning_content`, tool calls, and web-search results arrive over time. The system preserves the incoming text and execution events, incrementally identifies complete reasoning units, and builds a provisional graph while generation continues. Each completed unit receives a node category and incoming relationships from relevant earlier nodes. Structured tool events supply a linked execution layer through ToolCall and ToolResult nodes; returned evidence remains Context. When the response is complete, the system reconciles segmentation, node categories, conclusions, and affected relationships against the completed interaction. The final approved legend defines the visual mapping for all 11 node types and 20 edge types. ReasoningFlow supplies the original discourse schema and offline annotation method; streaming annotation, operational nodes, and provenance relationships are the extensions developed in this chat. Their accuracy has not been established by the paper. [1, Sections 3–4 and Appendix C.1]

## 1. Design strategy

**Build the graph during generation, then reconcile it when the interaction is complete.** The system does not need to wait for the entire reasoning trace before identifying useful relationships. It does need to wait until enough text has arrived to form a meaningful unit. A complete calculation can be linked to its premises; a later reflection can be linked to the claim it evaluates; a correction can be linked to an earlier statement as soon as the correction is available.

The graph has two linked layers. The **semantic layer** describes the relationships expressed in reasoning and answer text. The **execution and provenance layer** records tool requests, returned results, and connections between external evidence and later statements. Both layers appear in the integrated view, but they retain different evidential meanings. A recorded search establishes that a search occurred. An inferred grounding link indicates that a statement appears to rely on retrieved evidence. Neither establishes the model's internal computational path.

The processing strategy has three parts: capture the available interaction without blocking chat delivery; annotate completed units and execution events incrementally; reconcile the accumulated graph after generation ends. Expensive semantic annotation can remain inactive until the user opens the reasoning view. Opening that view during generation initiates backfill over the accumulated text and then follows the live stream.

This document describes that strategy from the chat and the attached ReasoningFlow paper. It does not prescribe a particular model, database, user-interface library, or provider-specific event contract. The existing annotation-guide document remains the detailed companion reference for the new labels. The visual reference is the final approved legend reproduced in Section 7; earlier legend drafts are superseded.

# 2. How the ReasoningFlow algorithm works

## 2.1. Represent the trace as a directed acyclic graph

ReasoningFlow converts a linear reasoning trace into a directed acyclic graph, or DAG. Nodes represent contiguous, non-overlapping text units. A unit is normally a sentence, but a sentence may contain several units when its clauses perform different functions. Edges connect earlier nodes to later nodes and identify their semantic relationships. One node may depend on several earlier nodes; the representation is therefore a graph rather than a single-parent tree. [1, Section 3]

The paper defines eight generated-text categories: Planning, Fact, Reasoning, Restatement, Assumption, Example, Reflection, and Conclusion. Context represents supplied information, including the problem statement, retrieved documents, and tool responses. This produces nine original node types when Context is counted. The 14 original edge labels fall into Reason, Plan, Reflect, and Validate families. [1, Appendix A, Tables 3–4]

Figure 1 illustrates the distinction between chronological order and semantic structure. The calculation proceeds through several nodes, then the trace questions the result and plans verification. These later steps add relationships to earlier material without moving the earlier text or introducing a backward-directed edge.

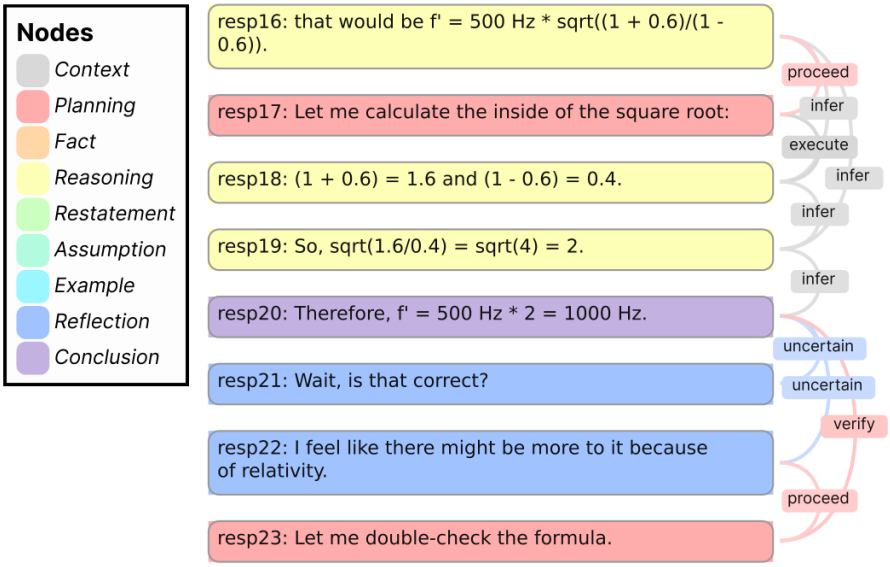



*Figure 1: Original ReasoningFlow graph: calculation, self-reflection, and a plan to verify an earlier result. Reproduced from Figure 1 of the supplied paper.*

*Source: Lee et al., ReasoningFlow, Figure 1, page 1. [1]*

## 2.2. Segment the text into reasoning units

The authors first apply rule-based coarse segmentation and then ask an LLM to divide those chunks into atomic reasoning units. Their coarse rule does not split at every double newline. It requires the preceding paragraph to end in a period and the next paragraph to begin with a capital letter, reducing the risk of splitting one semantic unit across an equation block. [1, Appendix C.1]

The finer segmentation identifies functional changes within a sentence. For example, “The answer is 17, but I should double-check” contains an answer statement and a verification plan. An assumption followed by its consequence can also require a split. Conversely, the paper keeps some constructions together, including a single LaTeX equation block and a sentence in

which the premise follows the conclusion. Its segmentation prompts provide the detailed exceptions. [1, Appendix I, Prompts 1–2]

The output must remain aligned with the original text. The authors report that an annotating LLM can alter whitespace or mathematical symbols; their implementation uses dynamic programming to recover alignment when returned segments do not exactly match the source. For this design, the important principle is to preserve source text and span offsets rather than treat the annotator's rewritten output as the authoritative trace. [1, Appendix C.1]

### 2.3. Classify nodes and identify conclusions

In the paper's offline pipeline, the classifier considers all segmented nodes together. It first assigns labels excluding Conclusion. A separate post-hoc pass then identifies nodes that should be reclassified as Conclusion. This addresses cases in which the first classification pass omits an answer node. Conclusion includes asserted intermediate answers as well as the final answer; it does not mean only the last sentence. [1, Appendix C.1; Appendix A, Table 3]

This batch context is one reason the streaming adaptation must permit revisions. An early answer can later become an intermediate answer. A statement that initially looks like a general reasoning step may be easier to interpret after the surrounding attempt is complete. The proposed incremental classifier therefore produces provisional categories rather than claiming to reproduce the offline classification immediately.

### 2.4. Find incoming edges for each destination node

For each destination node, the edge annotator selects the relevant earlier source nodes and assigns relationship labels. The objective is a minimal complete predecessor set: include the information needed to explain the destination step, while avoiding unnecessary links. The paper chooses this per-destination approach instead of annotating the entire edge set at once or making a separate decision for every possible node pair. [1, Appendix C.1]

The destination's type restricts the permitted incoming edge labels. For example, `positive`, `uncertain`, and `negative` lead into Reflection nodes. The annotator may return no predecessors when no relationship satisfies the definitions. Consecutive text does not, by itself, justify a semantic edge. [1, Appendix C.1; Appendix I, Prompt 4]

The graph permits quadratically many earlier-to-later pairs, but the authors observe an average of about 1.3–2.0 incoming edges per node in their dataset. That observation motivates selective predecessor search; it is not a rule limiting a node to two predecessors. [1, Sections 4.2.2 and 5.1]

## 3. Incremental classification while reasoning_content streams

### 3.1. Wait for a complete unit, not for the complete response

The stream collector appends each `reasoning_content` fragment to the original text buffer. It does not ask the annotator to classify every token or arbitrary transport chunk. A chunk boundary can occur in the middle of a word, sentence, equation, or tool argument. The segmenter instead holds the unfinished tail until it can identify a sufficiently complete unit.

The chat proposed buffering sentences or clauses and annotating small groups of completed units. A starting experiment is a microbatch of roughly three to five units, rather than one annotation request for every fragment. This is a proposed tuning choice, not a parameter established by ReasoningFlow. The batch size and any additional look-ahead should be adjusted to the observed annotation delay and revision rate.

A timeout alone does not prove that a semantic unit is complete. Text that ends with "because," an unfinished equation, or an open code block should remain pending or explicitly provisional. At a genuine completion boundary, the system flushes the remaining text and determines whether it forms a usable unit.

## 3.2. Classify the new unit using available context

Once a unit is ready, the classifier assigns its functional role using the prompt, relevant earlier text, and the current completed microbatch. The node retains its exact source span, message or turn identity, and a provisional annotation state. A candidate answer can receive the existing Conclusion category provisionally; “candidate” is a status, not an additional node type.

The classifier should distinguish what the statement does. “I should search for a source” is Planning. A paraphrase of retrieved text is Restatement. A deduction that combines retrieved claims is Reasoning. “I am not certain this is correct” is Reflection. These distinctions follow the original categories, while assigning them before the full trace is available is the proposed adaptation. [1, Appendix A, Table 3]

A label can become more stable after subsequent units arrive. This does not require continuously rewriting the entire graph. The system can keep older annotations, revisit ambiguous recent nodes, and reserve broader reconsideration for the completion pass. Earlier nodes must remain addressable because later units may refer to them.

## 3.3. Add incoming relationships from earlier nodes

A new destination node provides a natural incremental work item. The edge annotator searches earlier nodes, selects the necessary predecessors, and assigns the appropriate labels. Under fixed segmentation and labels, the graph update adds the new node and its incoming edges to the existing graph.

**Earlier nodes can gain new outgoing edges as later material arrives.** The chronological constraint means that an edge runs from an earlier node to a later node; it does not mean an earlier node’s full set of outgoing relationships is already known. A correction arriving much later can add an `attack` edge from the original claim to the new correction without reversing the direction of time.

The DAG structure makes incremental construction possible, but it does not guarantee that prefix-only annotation equals full-trace annotation. The paper classifies nodes jointly, and those categories affect edge choices. New context can also clarify a plan’s role or a statement’s scope. Incremental edges are therefore provisional semantic judgments unless their basis is a recorded execution relationship. [1, Section 3 and Appendix C.1]

## 3.4. Relationships that can be identified immediately

In this table, “immediately” means after the relevant destination unit is complete and the needed earlier information is available. It does not imply zero processing delay or an irreversible annotation.

| Relationship | Earliest useful decision point | What may still change |
|---|---|---|
| `infer` | A new derivation identifies information from earlier premises. | Predecessor selection or the node’s category may need correction. |
| `execute` | A completed reasoning step carries out an earlier plan. | The applicable plan may become clearer; this is not a tool invocation. |
| `restate`, `elaborate-fact`, `exemplify` | A completed unit paraphrases, details, or illustrates earlier content. | The original source or functional category may be ambiguous. |
| `proceed`, `decompose`, `backtrack` | A new plan or assumption establishes progression, decomposition, or an alternative direction. | The relationship between plans may need later disambiguation. |
| `verify` | A plan explicitly proposes checking an earlier claim. | The claim being checked may initially be unclear. |

| Relationship | Earliest useful decision point | What may still change |
|---|---|---|
| `positive`, `uncertain`, `negative` | A reflection evaluates identifiable earlier statements. | The target of the reflection may require revision. |
| `support`, `attack` | A later completed statement independently agrees with or contradicts an earlier claim. | The relevant earlier claim may be distant; final-answer relationships may be added later. |

These timings are design judgments derived from the label definitions and chronological graph structure, not streaming results reported by the authors. The labels themselves retain their ReasoningFlow meanings. [1, Appendix A, Table 4]

### 3.5. Search beyond the most recent nodes

The predecessor candidate set should combine nearby nodes with relevant older material. The chat identified active plans, assumptions, prior conclusions, and semantically related statements as useful long-range candidates. A recency-only window can miss a return to an earlier approach, a restatement of an old premise, or a contradiction of an early answer.

Candidate retrieval narrows the annotation task; it does not establish an edge. The annotator still decides whether a specific semantic relationship exists. The system should not connect every pair of similar statements, force an edge to the immediately preceding node, or treat the graph's observed average degree as a hard limit.

## 4. Integrating tool calls and web search

### 4.1. Capture execution events separately from reasoning text

The input to this design includes exposed `reasoning_content`, the assistant's answer text, and available structured tool-call and tool-result events. The collector preserves their association with the same interaction. It obtains execution information from those events rather than inferring that a tool ran because the model said it intended to use one.

A plan to search and a recorded search request are different nodes. A tool call may also occur without an exposed planning sentence. In that case, the graph can show the ToolCall and its result without inventing a Planning node or an `invokes` link.

The paper already classifies retrieved documents and tool-response text as Context. The extension adds two operational node types to represent the request and return explicitly; it does not replace the original Context category. [1, Appendix A, Table 3]

### 4.2. Use ToolCall, ToolResult, and Context together

**ToolCall** represents a structured request to an external tool. Its content identifies the tool and the available arguments. Web search is a retrieval subtype of ToolCall, alongside document search. Computation and external action are the other operational subtypes discussed in the chat. These are not additional top-level node types.

**ToolResult** represents the returned payload, status, or execution error associated with a call. It can contain search hits, retrieved page content, calculation output, or another tool response. The node preserves the operational result even when the model does not subsequently use it.

**Context** represents the returned information that is available as evidence or input to later reasoning. A ToolResult can provide several Context spans, such as separate search hits or excerpts from a retrieved document. The result envelope and the evidence spans have distinct purposes; they can reference the same underlying payload without displaying the full text twice.

The integrated sequence is therefore: a Planning node may motivate a ToolCall; the call returns a ToolResult; the result provides Context; later reasoning or answer nodes refer to that Context. The source text retains its provenance through the result and call. URLs, source identifiers, query strings, timestamps, and original snippets belong with that provenance.

## 4.3. Add the six relationships from the final legend

The following directions match the approved legend. For `grounded_in`, `cites`, and `retry_of`, the later node bears the named relation to the earlier reference. Thus a Context-to-statement `cites` arrow means that the later statement cites the earlier Context.

| Added label | Earlier source → later destination | When and how to identify it |
|---|---|---|
| `invokes` | Planning → ToolCall | After the structured call is identifiable. The plan-to-call association is inferred unless the runtime explicitly records that link. |
| `returns` | ToolCall → ToolResult | When the result is associated with its call. Use the recorded request/response association. |
| `provides` | ToolResult → Context | When a returned payload supplies an identifiable evidence span or context item. Preserve its source mapping. |
| `grounded_in` | Context → later statement | After a statement is complete and evidence dependence can be assessed. This is an inferred relationship, not proof of internal causation. |
| `cites` | Context → citing statement | When an explicit source reference can be resolved to its Context. Attribution does not establish that the source supports the statement. |
| `retry_of` | Earlier ToolCall → retry ToolCall | When a later call is identified as a retry of the earlier operation. Prefer explicit execution lineage; similar queries alone do not establish a retry. |

Tool arguments may arrive in fragments. The system can display a pending call while accumulating them, but it should not treat partial arguments as a complete request or execute a displayed call from the annotation process. Likewise, a result can be recorded as soon as it returns, while the reasoning that uses it remains unclassified until its text is complete.

## 4.4. Keep evidence provenance distinct from reasoning semantics

A retrieved passage is Context, not automatically Fact. In ReasoningFlow, Fact refers to general knowledge not supplied by the preceding context. A model's paraphrase of a passage can be Restatement; an inference based on that passage can be Reasoning. [1, Appendix A, Table 3]

The original `execute` relationship connects a plan to a reasoning step that carries it out. The new `invokes` relationship connects a plan to a tool request. The original `support` relationship captures agreement between independently derived propositions; it is not a replacement for `cites` or `grounded_in`. [1, Appendix A, Table 4]

The semantic and provenance layers may connect the same nodes for different reasons. A passage can be a premise of a derivation through `infer`, while also providing an evidence reference through `grounded_in`. The integrated view should retain that distinction rather than replace one label with the other. A tool retry should likewise remain distinct from reasoning `backtrack`: retrying a failed request does not necessarily indicate a new reasoning strategy.

# 5. Reconciliation when generation ends

## 5.1. Use completion of the interaction as the trigger

A pause in `reasoning_content` is not sufficient to finalize the graph. Reasoning may lead into a tool call, resume after the result, or be followed by answer text. The completion pass should operate on the completed response and its associated tool activity, not merely on the end of one reasoning segment.

If the reasoning view is active, reconciliation begins after the completion boundary is known. If the view is inactive and expensive annotation has been deferred, the system records that the interaction is complete and performs the same reconciliation when the graph is requested. This preserves lazy computation without changing the intended final analysis.

## 5.2. Finish segmentation and repair source alignment

The final pass first processes the remaining text and reviews uncertain boundaries. It confirms that nodes correspond to original source spans, including any final answer text. If a provisional node must be split or merged, the graph preserves the relationship between the revised spans and their prior representation.

Stable source references matter because relationships and selections may already point to the displayed node. Reconciliation should repair those references rather than silently discard text or leave edges attached to obsolete spans. The graph can change its interpretation while the underlying captured interaction remains unchanged.

## 5.3. Review categories and conclusions

The classifier then reconsiders ambiguous node categories with the completed trace available. It identifies the asserted answers across attempts and determines which answer is final. This follows the purpose of the paper's post-hoc Conclusion update, while applying it to an interaction that includes reasoning text, final answer text, and tool events. [1, Appendix C.1]

Earlier candidate conclusions should remain visible as intermediate conclusions when the trace later revises them. A later correction does not require rewriting the earlier statement as though it had always been correct. The graph records that the earlier answer was proposed and subsequently challenged.

The paper's procedure ensures a Conclusion in its completed benchmark traces. This design should not manufacture a conclusion in an interrupted interaction or in a `reasoning_content` segment that does not contain one. When available, the assistant's separate final answer provides the relevant conclusion text. An absent or incomplete answer should remain identified as such.

## 5.4. Revisit affected relationships

A changed boundary or node category can invalidate both the node's incoming edges and later edges that use it as a source. The completion pass revisits those affected decisions, checks type compatibility, and searches for missed long-range relationships. It also compares final and earlier conclusions where `support` or `attack` applies.

This is selective reconciliation, not an instruction to recompute every edge after every streamed fragment. Unchanged recorded call/result associations can remain intact. However, a selective repair pass is not guaranteed to match a fresh offline annotation. A full-trace rerun remains a useful comparison when evaluating whether the incremental method has missed broader relationships.

Full-trace annotation must also preserve historical availability. Later text can clarify an earlier statement's role, but a document returned later must not be represented as evidence available to an earlier generation step. New evidence can support a later correction; it cannot retroactively establish that the model used that evidence before retrieval.

### 5.5. Publish the reconciled graph without erasing its history

The final graph includes the retained reasoning units, execution nodes, and their revised relationships. It distinguishes observed execution associations from inferred semantic or grounding links. A finalized annotation means that the completion pass has finished; it does not mean that every statement is true or every inferred relationship is certain.

Cancelled or failed interactions retain their captured text and tool history, with incompleteness made explicit. Normal completion permits final-answer analysis. In both cases, provisional versus finalized status remains separate from node type and line style.

## 6. Worked example: search, verification, and correction

The following interaction is illustrative; ExampleService and its limits are fictional. The user asks which request limit a client should use. A search snippet suggests one value, but opening the documentation reveals another. The example shows when the graph can grow and what remains for reconciliation.

| Order | Node type | Available content or event |
|---|---|---|
| 1 | Planning | “I should search for the documented request limit.” |
| 2 | ToolCall | Search for “ExampleService request limit.” |
| 3 | ToolResult | Search results include a snippet stating a limit of 10 requests per minute. |
| 4 | Context | The returned snippet: “Limit: 10 requests per minute.” |
| 5 | Conclusion | “The client should use 10 requests per minute.” This is a provisional candidate answer. |
| 6 | Planning | “I should verify the limit in the full documentation.” |
| 7 | ToolCall | Open the documentation identified in the search results. |
| 8 | ToolResult | The documentation text returns. |
| 9 | Context | The returned documentation: “The current limit is 20 requests per minute.” |
| 10 | Reasoning | “The full document specifies 20, so the earlier estimate of 10 is not the current limit.” |
| 11 | Conclusion | “Use 20 requests per minute,” with an explicit reference to the documentation. |

At order 2, the system can create the ToolCall and assess `invokes` from node 1. At orders 3–4, it can record `returns` from 2 to 3 and `provides` from 3 to 4. Once node 5 is complete, it can provisionally classify the candidate answer and assess its relationship to the snippet. None of those decisions requires the later documentation to exist.

At order 6, the plan to check the candidate answer allows `verify` from node 5 to node 6. The second call and result add another execution chain. At order 10, the system can identify a correction and add `attack` from node 5 to node 10. This is an incremental relationship: the earlier answer is challenged as soon as the later correction is available, not only after the response ends.

At order 11, the system can add `infer` from the correction to the answer, assess `grounded_in` from Context node 9, and record `cites` when the documentation reference resolves. An `attack` relationship from the earlier candidate conclusion to the later conclusion is also available once that later statement is complete. These relationships express different aspects of the same interaction and should not be collapsed into a generic “related” edge.

At completion, reconciliation confirms node 5 as an intermediate conclusion and node 11 as the final conclusion, reviews the correction's premises, and checks that the source links resolve. The earlier answer remains part of the trace. The final graph should show both the provisional decision and the evidence-driven revision.

A failed search would produce a ToolResult carrying the failure rather than a fabricated evidence passage. A subsequent repeated operation could have `retry_of` from the earlier call to the retry when that relationship is established. An alternative search strategy may instead correspond to a new plan and reasoning `backtrack`; the two cases should not be conflated.

## 7. Final legend and visual mapping

The approved legend is the visual reference for this design. It contains 11 node types and 20 edge types. All nodes, including ToolCall and ToolResult, use rectangular boxes with the same rounded-corner treatment. Semantic edge labels appear inside colored boxes, accompanied by matching colored lines. The top-left label reads "Legend."

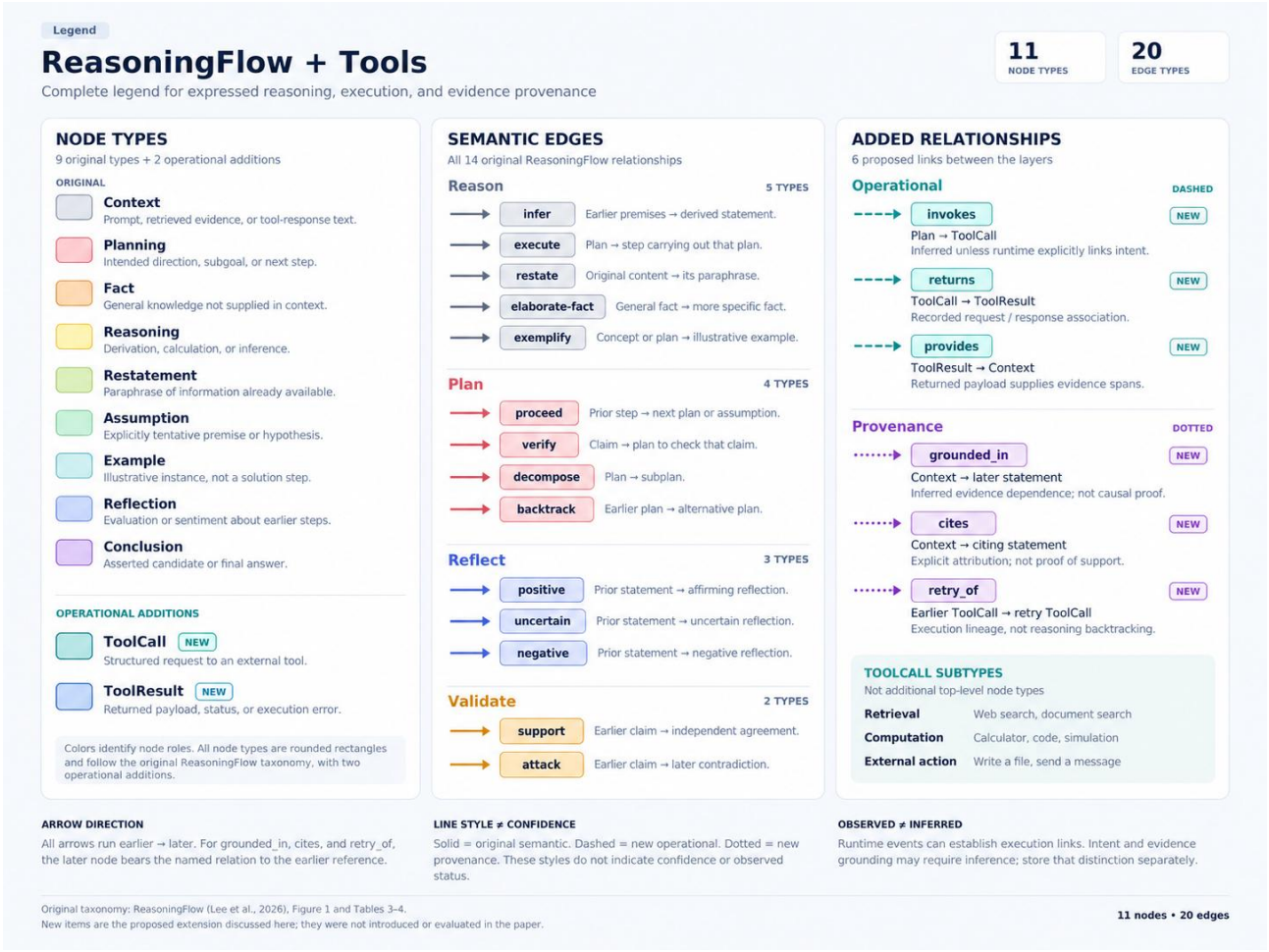



*Figure 2: Final approved Legend. All node types remain boxes; original semantic labels use colored boxes and solid lines, operational additions use dashed teal lines, and provenance additions use dotted purple lines.*

The final diagram distinguishes dashed operational links from dotted provenance links. That specific mapping takes precedence over earlier, broader suggestions to make every added relationship dashed. Neither dash pattern nor color indicates confidence. The same label and visual treatment should be used in the legend, graph, and node or edge details.

## 7.1. Node colors

The named colors below map to the swatches in Figure 2. The figure is the reference for the selected shades; the names describe their roles rather than introducing a different palette.

| Node label | Final legend color | Meaning in the integrated view |
|---|---|---|
| Context | Light gray | Supplied prompt text, retrieved evidence, or tool-response text. |
| Planning | Pink | Intended direction, subgoal, or next step. |
| Fact | Peach / orange | General knowledge not supplied by preceding context. |
| Reasoning | Pale yellow | Derivation, calculation, or inference. |
| Restatement | Light yellow-green | Paraphrase of information already available. |
| Assumption | Mint green | Explicitly tentative premise or hypothesis. |
| Example | Light cyan | Illustrative instance. |
| Reflection | Light blue | Evaluation or sentiment about earlier steps. |
| Conclusion | Lavender | Asserted candidate or final answer. |
| ToolCall | Teal | Structured request to an external tool. |
| ToolResult | Blue | Returned payload, status, or execution error. |

## 7.2. Edge colors and line styles

| Edge family | Labels | Final legend treatment |
|---|---|---|
| Reason | `infer`, `execute`, `restate`, `elaborate-fact`, `exemplify` | Gray label boxes and solid gray lines. |
| Plan | `proceed`, `verify`, `decompose`, `backtrack` | Pink label boxes and solid pink/red lines. |
| Reflect | `positive`, `uncertain`, `negative` | Blue label boxes and solid blue lines. |
| Validate | `support`, `attack` | Orange label boxes and solid orange lines. |
| Operational | `invokes`, `returns`, `provides` | Teal label boxes and dashed teal lines. |
| Provenance | `grounded_in`, `cites`, `retry_of` | Purple label boxes and dotted purple lines. |

Arrows follow the earlier-to-later orientation shown in the legend. Reconciliation may discover a relationship later, but its direction still follows the order of the underlying nodes. The display should not reverse `cites`, `grounded_in`, or `retry_of` merely to make the label read as a conventional subject–verb phrase.

Annotation status should appear through a separate status indicator or detail field. The design should not change an original semantic edge to dashed because it is provisional; dashed lines already identify the operational family. Likewise, recorded versus inferred status should be explicit rather than implied by color.

# 8. Processing and view lifecycle

## 8.1. Keep graph annotation off the chat delivery path

The stream collector preserves the original text and structured events before expensive annotation. A separate annotation worker processes completed units and updates the graph. The chat can continue displaying content while that worker is still classifying earlier text. The view should show that the graph is catching up rather than imply that unprocessed text has no relationships.

The incremental worker maintains the source buffer, an unfinished tail, existing node references, and pending annotation work. It uses the current graph and candidate retrieval to avoid beginning from scratch for every new unit. Tool execution links can be captured from structured events without making an LLM call to decide whether the execution occurred.

## 8.2. Preserve lazy activation

| View state | Processing behavior |
|---|---|
| Reasoning view has not been opened | Save text and execution events. Defer expensive semantic annotation. |
| View opens while generation continues | Backfill accumulated completed units, then follow newly arriving units. |
| View remains open | Add provisional nodes and relationships as annotation finishes. |
| Generation completes with the view active | Flush remaining work and reconcile the completed interaction. |
| View opens after generation is complete | Annotate the recorded interaction and perform completion reconciliation. |
| View reopens with an unchanged graph | Use the cached graph rather than repeat unchanged annotation. |

The backfill and live-stream paths must refer to the same captured text and events so they do not create duplicate units. Existing nodes should retain their identity where possible. The visualization should update affected portions of the graph rather than unnecessarily rearranging the entire display after every annotation.

### 8.3. Processing outline

The following pseudocode describes the design flow. Its event names are conceptual, not names from a specific provider API.

```
On incoming text or tool event:
    Preserve the original event and its interaction identity.
    Append text or accumulate the structured call/result.
    Record identifiable execution associations.
    If semantic annotation is active:
        Find newly completed reasoning units.
        Classify the completed units provisionally.
        For each new destination:
            Retrieve relevant earlier candidates.
            Select and label its incoming relationships.
        Publish the provisional graph update.

On reasoning-view activation:
    Load the cached graph, if current.
    Backfill unprocessed captured content.
    Follow the live stream if generation is still active.
    Otherwise perform completion reconciliation.

On interaction completion:
    Record the completion boundary.
    If annotation is active:
        Flush the remaining text and pending annotation.
        Review segmentation, categories, and conclusions.
        Revisit relationships affected by those revisions.
        Publish the reconciled graph.
    Otherwise defer the same work until the view is opened.
```

### 8.4. Handle context across turns without inventing use

The graph should retain the distinction between user prompts, earlier messages, retrieved sources, and the current generation. Earlier material can become a predecessor candidate when it was available in the relevant interaction context. A retained conversation record does not, by itself, establish that a particular model invocation received or used every earlier message.

The design therefore preserves the available context association and avoids asserting evidence use solely because a source is present somewhere in the conversation. Missing reasoning text, unavailable tool events, and unresolved source references remain missing or unresolved. The annotator should not reconstruct an unobserved tool execution or supply hidden reasoning to make the graph appear complete.

## 9. Interpretation and evaluation limits

ReasoningFlow annotates the discourse structure expressed in text. The paper reports a gap between these language-level relations and mechanistically measured step-to-step dependencies. The integrated graph should therefore be described as a representation of expressed reasoning, observed execution, and inferred provenance, rather than as a verified account of the model's internal thought process. [1, Section 8]

The main implementation question is how much annotation can be performed early without creating excessive revisions or missing long-range relationships. The comparison proposed in the chat is to replay completed traces as streams and compare prefix-only annotation, incremental annotation with final reconciliation, and full-trace batch annotation. Relevant measures include node and edge agreement, annotation delay, computational cost, and the number of already displayed relationships that change.

No streaming accuracy, throughput, or latency result is claimed here. The practical design commitment is narrower: expose useful provisional structure when completed units and events become available, preserve uncertainty about that structure, and use the completed interaction to reconcile it. The final legend remains consistent throughout those stages.

## References

[1] Jinu Lee, Shivam Agarwal, Amruta Parulekar, Siddarth Madala, Dilek Hakkani-Tür, and Julia Hockenmaier. 2026. *ReasoningFlow: Discourse Structures for Understanding LLM Reasoning Traces*. arXiv:2606.05402v1, June 3, 2026. Source: the PDF supplied in this conversation. Algorithm: Sections 3–4 and Appendix C.1. Annotation definitions: Appendix A, Tables 3–4. Prompts: Appendix I. Interpretation limits: Section 8.

Design source: the discussion in this chat concerning streaming annotation, tool calls, web search, completion reconciliation, and the final approved “ReasoningFlow + Tools” legend. These additions are proposed design choices and are not presented as results or features validated by the ReasoningFlow authors.